

Introduction to Reinforcement Learning

Lecture 1: Markov Decision Process & Dynamic Programming

Mireille Sarkiss

Associate Professor

Telecom SudParis, IP Paris

November, 2024

Outline: Sequential Decision-Making Processing

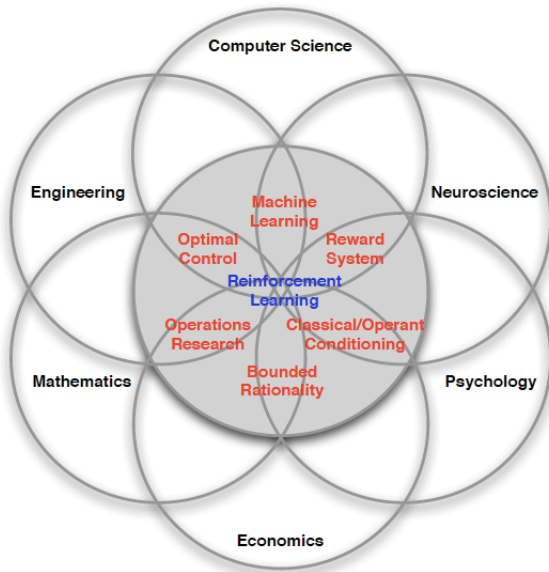
- 1 Introduction to Reinforcement learning
- 2 Review on Markov Decision Process (MDP) and Model-based RL methods
- 3 Model-free RL methods
- 4 Value Function approximation and Deep Reinforcement Learning

Teaching sessions

- 6 sessions: 27/11 - 20/12
- Lectures + Tutorial sessions + 1 lab
- 2 Quizzes : Sessions 3 & 6: 13/12 & 20/12
- Exam: 07/01, 09/01?

- 1- Reinforcement Learning
- 2- MDP and Model-based RL methods

Reinforcement Learning Application domains



Today's lectures

- 1 Sequential Decision Making
- 2 Markov Decision Process
- 3 Dynamic programming

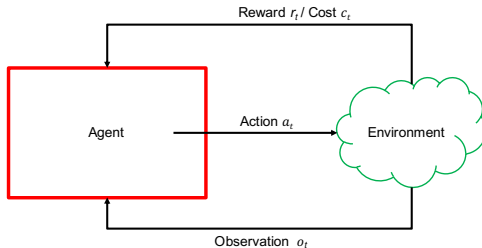
References:

- An Introduction to Reinforcement Learning, Sutton and Barto, 2nd Edition, 2018: Chapters 1, 3, 4
- Slides of Prof. David Silver, University College London: Lectures 1, 2, 3
- Slides of Prof. Emma Brunskill, Stanford University: Lectures 1, 2

Sequential Decision Making

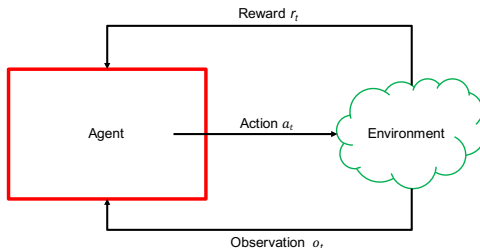
- Select sequence of decisions (actions) to optimize the goal of a problem (an objective function)
 - ▶ maximize a reward
 - ▶ minimize a cost
- Actions may have long-term consequences ➡ delayed rewards
- May require balancing immediate and long-term rewards
- May require a strategy to achieve high rewards
- **Discrete** time: At each step, decisions taken according to available information
- Additional information can be received between decisions
 - ➡ randomness in subsequent decisions making
- **Continuous** time: Decisions are replaced by a control, function of time
- Examples: Invent control, autonomous car driving, autopilot, robotics, playing tennis, chess, healthcare, portfolio management, resource allocation..

Agent and Environment



- Decision maker is called **agent** and interacts continually with **environment**
- At each time step t
 - ▶ Agent executes action a_t
 - ▶ Agent receives observation o_t and reward r_t
- Reward r_t is a scalar feedback signal defining how good or bad an event at time t is for the agent
- Agent aims to maximize total reward over long run
- Environment receives action a_t , emits observation o_{t+1} and reward r_{t+1}

History and State



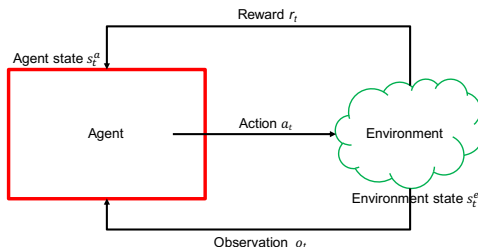
- History: sequence of past observations, actions and rewards up to time t

$$h_t = o_1, r_1, a_1, \dots, a_{t-1}, o_t, r_t$$

- Agent selects actions depending on history
- Environment responds with observations and rewards
- State is information used to determine what happens next
 - ▶ function of history

$$s_t = f(h_t)$$

Environment, Agent and Information States



- Environment state s_t^e : environment's private representation
 - ▶ information used to determine how environment generates next observation and reward
 - ▶ usually not visible or known to agent
 - ▶ even if known may contain irrelevant information to agent
- Agent state s_t^a : agent's internal representation
 - ▶ information agent uses to make next decision/action
 - ▶ any function of history (number of executed computations, number of decisions left ...)

$$s_t^a = f(h_t)$$

- Information state s_t : all useful information (statistics) from history

A state s_t is Markov if and only if

- Future is independent of past given present

$$h_{1:t} \rightarrow s_t \rightarrow h_{t+1:\infty}$$

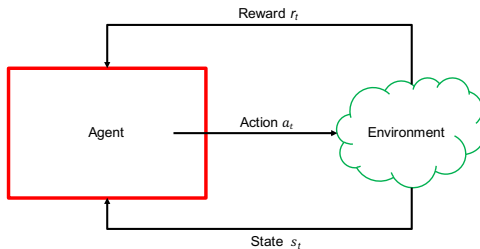
- Once state known, history may be thrown away

- ➡ State is sufficient statistic of future

- ▶ environment state is Markov
- ▶ history is Markov

- State representation affects: Computational complexity, data required $\Rightarrow$ resulting performance

Fully Observable Environments



- **Full observability:** Agent directly observes environment state

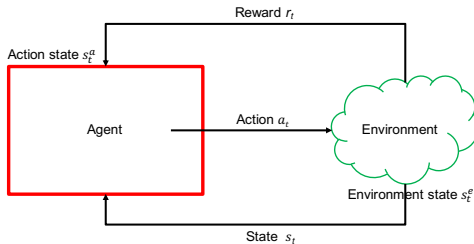
$$o_t = s_t^a = s_t^e$$

- Agent state = Environment state = Information state

$$s_t = o_t$$

➡ Markov Decision Process (MDP)

Partially Observable Environments



- **Partial observability:** Agent indirectly observes environment state
- Agent state $\neq$ Environment state: $s_t^a \neq s_t^e$
- ➔ Partially Observable Markov Decision Process (POMDP)
- Agent constructs its own state
 - Complete history: $s_t^a = h_t$
 - Beliefs of environment state: $s_t^a = (P(s_t^e = s^1), \dots, P(s_t^e = s^n))$
 - Recurrent neural network: $s_t^a = f(s_{t-1}^a, o_t)$
- Examples: Poker game, robot with camera vision

Types of Sequential Decision Processes

- MDPs and POMDPs
 - Actions influence future observations
 - Strategic actions may be needed

- Bandits: MDPs with one state
 - Actions have no influence on next observations
 - No delayed rewards

- How processes change?
 - ▶ **Deterministic**: Given history and action, single observation and reward (robotics, controls)
 - ▶ **Stochastic**: Given history and action, many potential observations and rewards (customers, patients)

Components of Sequential Decision Processes

- Agent can include one or more components
 - ▶ Policy: agent's behaviour function
 - ▶ Value function: how good is each state and/or action over long run when following particular policy
 - ▶ Model: agent's representation of environment, how it changes in response to agent's actions

Policy

- Policy is agent's behaviour at a given time
- Policy π determines how agent selects actions
- Mapping from a state to an action:

$$\pi : s \rightarrow a$$

- Deterministic policy:

$$\pi(s) = a$$

- Stochastic policy:

$$\pi(a|s) = P(a_t = a | s_t = s)$$

Value Function

- Value function v is prediction of future reward
- Used to evaluate goodness/badness of states → select between actions
- Following a policy π , value function is expected discounted sum of future rewards

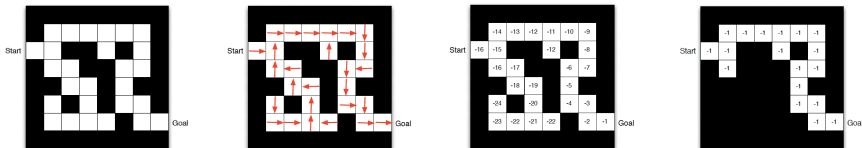
$$v_{\pi}(\mathbf{s}) = \mathbb{E}_{\pi} \left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | \mathbf{s}_t = \mathbf{s} \right]$$

- ▶ γ is discount rate, $0 \leq \gamma \leq 1$
 - ▶ γ weighs immediate vs future rewards
- Examples: simple function or lookup table, search process with extensive computation

- $$P_{ss'}^a = P(s_{t+1} = s' | s_t = s, a_t = a)$$

- $$R_s^a = R(s, a) = \mathbb{E}[r_{t+1} | s_t = s, a_t = a]$$

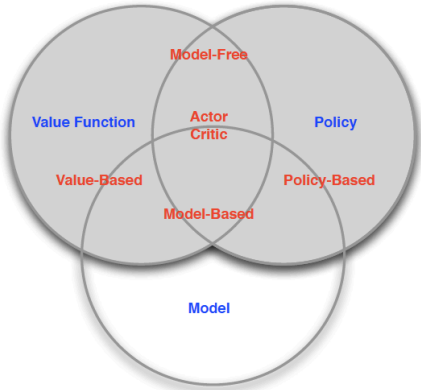
Maze Example



- Rewards: -1 per time-step
- Actions: N, E, S, W
- States: Agent's location
- Arrows represent policy $\pi(s)$ for each state s
- Numbers represent value $V_{\pi}(s)$ of each state s
- Grid represents transition/dynamics model $P_{ss'}^a$
- Numbers represent immediate reward R_s^a from each state s

Types of Agents

- Model-based
 - ▶ Explicit: Policy and/or value function
 - ▶ Model
- Model-free
 - ▶ Explicit: Policy and/or value function
 - ▶ No Model
- Value based or Policy-based
- Actor-critic
 - ▶ Explicit: Policy
 - ▶ Explicit: Value function



Sequential Decision Making Problems: Planning

- Model of environment is known (dynamics and reward model)
- Agent performs computations with its model without any interaction with real environment
- Agent improves its policy (maximize expected reward)
- Examples: Atari / Solitaire games: Single player
 - ▶ Known rules of game → Perfect model
 - ▶ With action a from state s , compute probability of next state and potential score
 - ▶ Plan ahead to decide optimal action and find optimal policy
 - ▶ E.g. Dynamic programming, tree search, . . .

- Environment is initially unknown
- Agent interacts with environment to implicitly/explicitly learn how it behaves
- Agent improves its policy: trial-and-error learning
- Exploitation and Exploration tradeoff:
 - ▶ **Exploitation**: select actions that are expected to maximize reward given past experience
 - ▶ **Exploration**: find more information about environment to maybe make better decisions in future
 - ▶ Sacrifice reward to explore and learn about potentially better policy
 - ▶ E.g.: Movies, Advertising, Restaurant Selection, Driving, Game playing
- Examples: Atari / Solitaire games: Single player
 - ▶ Unknown rules of game
 - ▶ Learn directly from interactive game-play
 - ▶ Try to find a good policy over time that yields high reward

Prediction and Control problems

- Prediction:
 - ▶ Evaluate the future given a policy
 - ▶ E.g. What is the value function for the uniform random policy?
- Control:
 - ▶ Optimization: Find the best policy
 - ▶ E.g. What is the optimal value function over all possible policies?
 - ▶ What is the optimal policy?

- ## Markov State

A state s_t is Markov if and only if

$$P(s_{t+1}|s_t) = P(s_{t+1}|s_1, \dots, s_t)$$

- Future is independent of past given present $\Rightarrow$ State is sufficient statistic of future
- For a Markov state s and successor state s' , state transition probability

$$P_{ss'} = P(s'|s) = P(s_{t+1} = s' | s_t = s)$$

- For finite number of states N , state transition matrix P defines transition probabilities from all states s to all successor states s'
- State transition matrix P where each row of the matrix sums to 1

$$P = [P_{ss'}] = \begin{bmatrix} P_{s_1 s_1} & P_{s_1 s_2} & \cdots & P_{s_1 s_N} \\ P_{s_2 s_1} & P_{s_2 s_2} & \cdots & P_{s_2 s_N} \\ \vdots & \vdots & \ddots & \vdots \\ P_{s_N s_1} & P_{s_N s_2} & \cdots & P_{s_N s_N} \end{bmatrix}$$

Markov Process

- Markov process is memoryless random process
 - Sequence of random states $s_1, s_2, \dots$ with Markov property

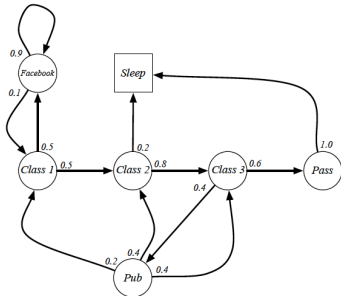
Markov Process or Markov Chain

A Markov Process (or Markov Chain) is a tuple $\langle S, P \rangle$

- S is a (finite) set of states ($s \in S$)
- P is state transition probability matrix with

$$P_{ss'} = P(s'|s) = P(s_{t+1} = s' | s_t = s)$$

Example:
Student Markov Chain



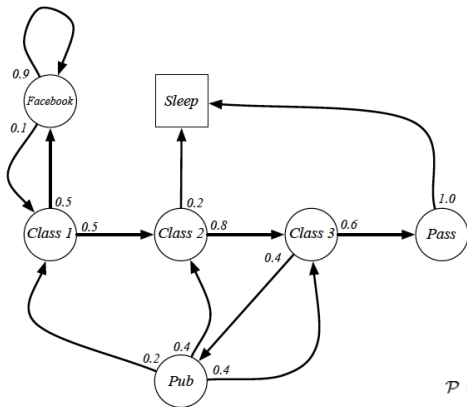
Example: Student Markov Chain: Episodes and Transition Matrix

- Sample **episodes** for Student Markov Chain starting from $s_1 = C1$

► Sequence of states:

$$s_1, s_2, \dots, s_T$$

- C1 C2 C3 Pass Sleep
- C1 FB FB C1 C2 Sleep
- C1 C2 C3 Pub C2 C3 Pass Sleep
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep



$$\mathcal{P} = \begin{matrix} & \begin{matrix} C1 & C2 & C3 & Pass & Pub & FB & Sleep \end{matrix} \\ \begin{matrix} C1 \\ C2 \\ C3 \\ Pass \\ Pub \\ FB \\ Sleep \end{matrix} & \begin{bmatrix} & & & & & & \\ & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ 0.2 & 0.4 & 0.4 & & & & 1.0 \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1 \end{bmatrix} \end{matrix}$$

- ## Markov Reward Process

- S is a (finite) set of states ($s \in S$)
- P is a state transition probability matrix with

$$P_{ss'} = P(s'|s) = P(s_{t+1} = s' | s_t = s)$$

- $$R_s = R(s_t = s) = \mathbb{E}[r_{t+1} | s_t = s]$$

- If finite number of states N , can express R as a vector

Return and Value Function

Horizon

Number of time-steps (T) in each episode

- Infinite ($T \rightarrow \infty$) or Finite Horizon (T) $\Rightarrow$ Infinite or finite Markov reward process

Return

The return G_t of an MRP is total discounted reward from time-step t to horizon

- Discounted sum of rewards from time-step t

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- Discount factor $\gamma \in [0, 1]$ is present value of future rewards
- Value of receiving reward r after $k + 1$ time-steps is $\gamma^k r$

State Value Function

The state value function $v(s)$ of an MRP gives long-term value of state s

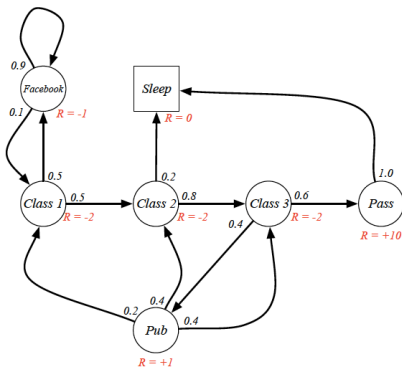
- Expected return starting from state s

$$v(s) = \mathbb{E}[G_t | s_t = s]$$

Discount Factor

- Discount factor γ evaluates immediate reward vs future delayed rewards
- $\gamma \approx 0$, more care about immediate reward ➡ “myopic evaluation”
- $\gamma \approx 1$, future reward almost as beneficial as immediate reward ➡ “far-sighted evaluation”
- Mathematically convenient to discount rewards: avoid infinite returns and values
- Human behaviour shows preference for immediate reward
 - ▶ Usually $\gamma < 1$: time limit to solve a task ➡ better to have rewards sooner rather than later
- Uncertainty about future may not be fully represented
- If all sequences terminate (episode lengths always finite), can use $\gamma = 1$ (*Undiscounted* Markov reward)

Example: Student MRP Returns

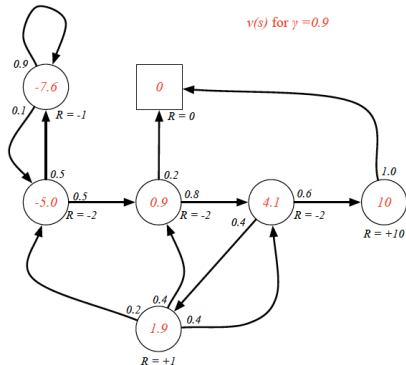
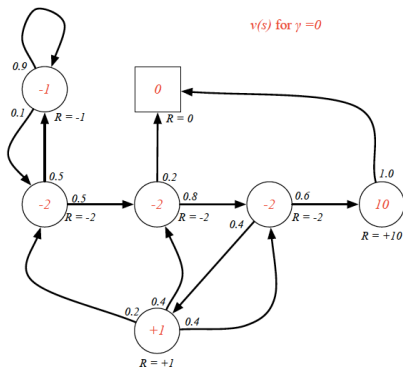


Sample returns for Student MRP starting from $s_1 = \text{C1}$ with $\gamma = \frac{1}{2}$

$$G_1 = r_2 + \gamma r_3 + \dots + \gamma^{T-2} r_T$$

- C1 C2 C3 Pass Sleep: $v_1 = -2 - 2 \times \frac{1}{2} - 2 \times \frac{1}{4} + 10 \times \frac{1}{8} = -2.25$
- C1 FB FB C1 C2 Sleep: $v_1 = -2 - 1 \times \frac{1}{2} - 1 \times \frac{1}{4} - 2 \times \frac{1}{8} - 2 \times \frac{1}{16} = -3.125$
- C1 C2 C3 Pub C2 C3 Pass Sleep:
 $v_1 = -2 - 2 \times \frac{1}{2} - 2 \times \frac{1}{4} + 1 \times \frac{1}{8} - 2 \times \frac{1}{16} - 2 \times \frac{1}{32} + 10 \times \frac{1}{64} = -3.41$
- C1 FB FB C1 C2 C3 Pub C1 FB FB FB C1 C2 C3 Pub C2 Sleep:
 $v_1 = -2 - 1 \times \frac{1}{2} - 1 \times \frac{1}{4} - 2 \times \frac{1}{8} - 2 \times \frac{1}{16} - 2 \times \frac{1}{32} + 1 \times \frac{1}{64} - 2 \times \frac{1}{128} + \dots = -3.2$

Example: Student MRP State Value Function

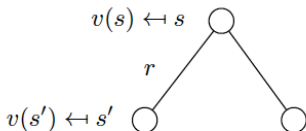


Computing State Value Function

- Can estimate by simulation
 - Generate a large number of episodes
 - Average returns
 - Bound how quickly average concentrates to expected value
 - No assumption of Markov structure
- ⇒ Markov property gives additional structure

- $$\begin{aligned} v(s) &= \mathbb{E}[G_t | s_t = s] \\ &= \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s] \\ &= \mathbb{E}[r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) | s_t = s] \\ &= \mathbb{E}[r_{t+1} + \gamma G_{t+1} | s_t = s] \end{aligned}$$

- $$v(s) = \mathbb{E}[r_{t+1} + \gamma v(s_{t+1}) | s_t = s]$$

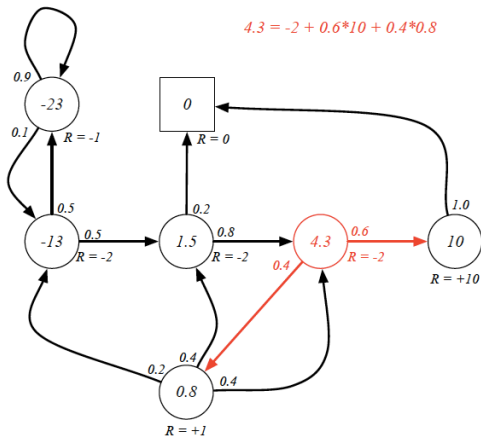


$$v(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s)v(s')$$

$$v(s) = R_s + \gamma \sum_{s' \in S} P_{ss'} v(s')$$

Example: Bellman Equation for Student MRP

$$v(s) = R_s + \gamma \sum_{s' \in S} P_{ss'} v(s')$$



- $$\begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_N) \end{bmatrix} = \begin{bmatrix} R_{s_1} \\ R_{s_2} \\ \vdots \\ R_{s_N} \end{bmatrix} + \gamma \begin{bmatrix} P_{s_{11}} & P_{s_{12}} & \dots & P_{s_{1N}} \\ P_{s_{21}} & P_{s_{22}} & \dots & P_{s_{2N}} \\ \vdots & \vdots & \ddots & \vdots \\ P_{s_{N1}} & P_{s_{N2}} & \dots & P_{s_{NN}} \end{bmatrix} \begin{bmatrix} v(s_1) \\ v(s_2) \\ \vdots \\ v(s_N) \end{bmatrix}$$

$$\begin{aligned} v &= R + \gamma P v \\ (I - \gamma P) v &= R \\ v &= (I - \gamma P)^{-1} R \end{aligned}$$

- 34 / 72

- ## Markov Decision Process

- S is a (finite) set of states ($s \in S$)
- A is a (finite) set of actions ($a \in A$)
- P is a state transition probability matrix for each action with

$$P_{ss'}^a = P(s'|s, a) = P(s_{t+1} = s' | s_t = s, a_t = a)$$

- R is a reward function with

$$R_s^a = R(s_t = s, a_t = a) = \mathbb{E}[r_{t+1} | s_t = s, a_t = a]$$

- γ is a discount factor, $\gamma \in [0, 1]$

Policy

A policy π is a distribution over actions given states

$$\pi(a|s) = P(a_t = a | s_t = s)$$

- Deterministic policy or stochastic policy
- Fully define agent behaviour
- MDP policies depend on current state not history
 - Policies are stationary i.e. time-independent
 $a_t \sim \pi(\cdot | s_t) \quad \forall t \geq 0$
- Given MDP $\langle S, A, P, R, \gamma \rangle$ and policy π
- State and reward sequence $s_1, r_1, s_2, r_2, \dots$ is Markov reward process

MRP $\langle S, P^\pi, R^\pi, \gamma \rangle$ where

$$P_{ss'}^{\pi} = \sum_{a \in A} \pi(a|s) P_{ss'}^a$$

$$R_s^\pi = \sum_{a \in A} \pi(a|s) R_s^a$$

Value Function

State Value Function

The state value function $v_\pi(s)$ of an MDP is expected return starting from state s and following policy π

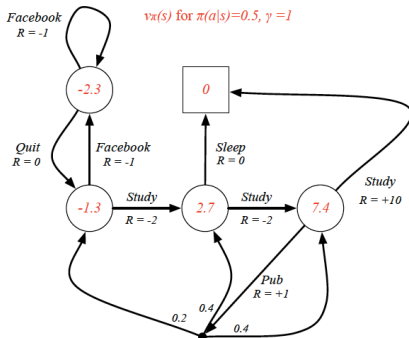
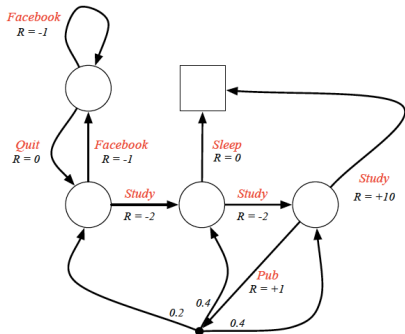
$$v_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

State-Action Value Function

The state-action value function $q_\pi(s, a)$ of an MDP is expected return starting from state s , taking action a and following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

Example: Student MDP and State Value Function

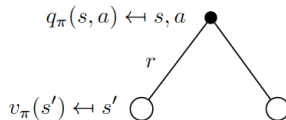


- $$v_{\pi}(\mathbf{s}) = \mathbb{E}_{\pi}[r_{t+1} + \gamma v_{\pi}(\mathbf{s}_{t+1}) | \mathbf{s}_t = \mathbf{s}]$$

- $$q_{\pi}(s, a) = \mathbb{E}_{\pi}[r_{t+1} + \gamma q_{\pi}(s_{t+1}, a_{t+1}) | s_t = s, a_t = a]$$

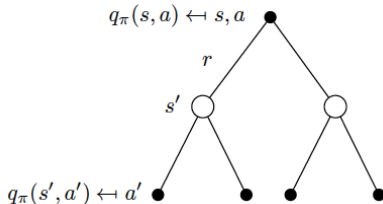
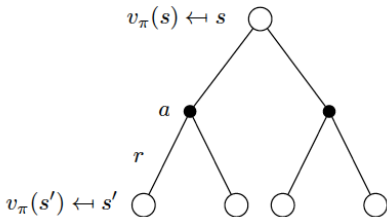
-
- A diagram of a simple tree structure. The root node is a white circle at the top. It has two children, both represented by black circles. The left child is labeled $q_\pi(s, a) \leftarrow a$ and the right child is labeled $v_\pi(s) \leftarrow s$.

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) q_{\pi}(s, a)$$



$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_{\pi}(s')$$

Bellman Expectation Equation for v_π and q_π



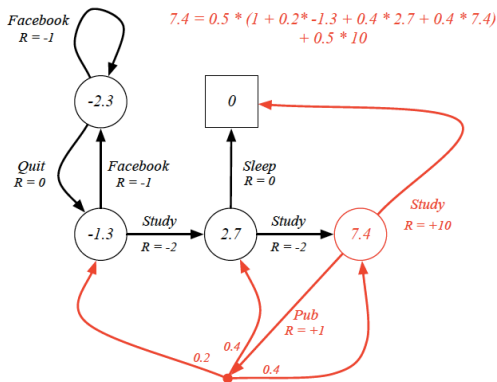
$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_\pi(s') \right)$$

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') q_\pi(s', a')$$

Example: Bellman Expectation Equation for Student MDP

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s') \right)$$

$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') q_{\pi}(s', a')$$



Bellman Expectation Equation for MDP: Matrix Form and Solution

- Bellman Expectation Equation can be expressed using MRP

$$v_{\pi} = R^{\pi} + \gamma P^{\pi} v_{\pi}$$

- Direct solution

$$v_{\pi} = (I - \gamma P^{\pi})^{-1} R^{\pi}$$

Optimal Value Function

Optimal Value Function

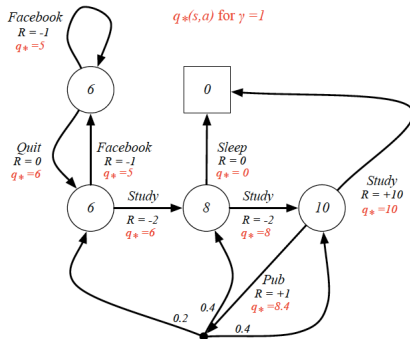
The optimal state value function $v_*(s)$ is the maximum state-value function over all policies

$$v_*(s) = \max_{\pi} v_{\pi}(s)$$

The optimal state-action value function $q_*(s, a)$ is the maximum action-value function over all policies

$$q_*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- Optimal value function specifies best possible performance of MDP
- MDP is solved when finding optimal value function



Optimal Policy

- Define partial ordering over policies

$$\pi \geq \pi' \text{ if } v_{\pi}(s) \geq v_{\pi'}(s) \forall s$$

Theorem

For any Markov Decision Process

- There exists an optimal policy π_* that is better than or equal to all other policies

$$\pi_* \geq \pi \forall \pi$$

- All optimal policies achieve optimal state value function

$$v_{\pi_*}(s) = v_*(s)$$

- All optimal policies achieve optimal state-action value function,

$$q_{\pi_*}(s, a) = q_*(s, a)$$

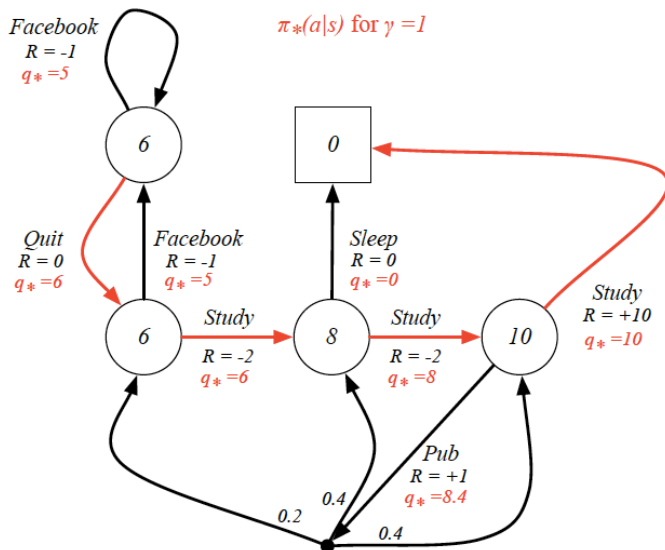
Finding Optimal Policy

- Find optimal policy π_* by maximizing over optimal state-action value function $q_*(s, a)$
 - ▶ Solve $q_*(s, a)$
 - ▶ Select action that gives most optimal $q_*(s, a)$

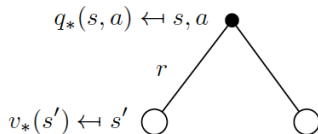
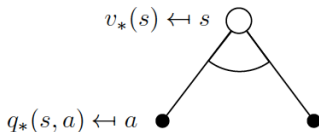
$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in A} q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- There is always deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, get directly optimal policy

Example: Optimal Policy for Student MDP

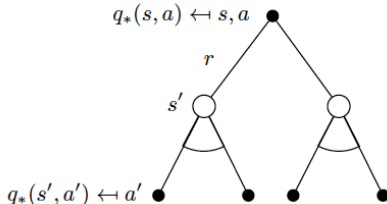
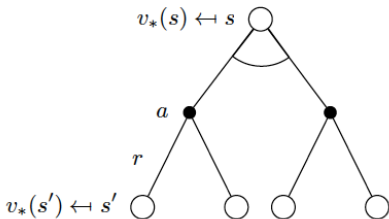


Bellman Optimality Equation for v_* and q_* (1)



$$v_*(s) = \max_{a \in A} q_*(s, a)$$

$$q_*(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s')$$



$$v_*(s) = \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s') \right)$$

$$q_*(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a' \in A} q_*(s', a')$$

Extensions of MDP

- Infinite and continuous MDPs
 - ▶ Countably infinite state and/or action spaces
 - ▶ Continuous state and/or action spaces
 - ▶ Continuous time (requires partial differential equations)
- Partially Observable MDPs
 - ▶ MDP with hidden states
 - ▶ Use belief state: probability distribution over states conditioned on history
 - ▶ Belief state satisfying Markov property
- Undiscounted Average reward MDPs
 - ▶ Ergodic Markov process (recurrent and aperiodic) has limiting stationary distribution
 - ▶ Ergodic MDP (Markov process for any policy is ergodic) has average reward per time step independent of start state
 - ▶ Value function expressed in terms of average reward

Planning Dynamic Programming

- Dynamic programming
 - ▶ **Dynamic** sequential or temporal component to the problem
 - ▶ **Programming** optimising a “program” ➡ find the best policy
- Mathematical and algorithmic method solving complex optimization problems
- Paradigm proposed by Richard Bellman in 1953 at RAND Corporation
- Principle: Breaking the complex problem down into subproblems
 - ▶ Calculate optimal solutions to the subproblems and memorise them
 - ▶ Combine solutions to subproblems
 - ▶ Calculate the optimal solution to the initial problem

Dynamic Programming

Dynamic Programming: general solution method for problems which 2 properties

- Optimal substructure
 - ▶ Principle of optimality
 - ▶ Optimal solution can be decomposed into subproblems
- Overlapping subproblems
 - ▶ Subproblems recur many times
 - ▶ Solutions can be cached and reused
- MDPs satisfy both properties
 - ▶ Bellman equation gives recursive decomposition
 - ▶ Value function stores and reuses solutions

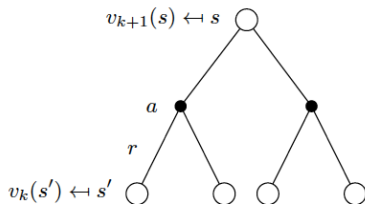
Prediction and Control Problems

- Dynamic programming assumes perfect model of environment as MDP ➡ full knowledge of MDP
- Used for Planning in MDP
- Prediction (Evaluation) Problem
 - ▶ Input: MDP $\langle S, A, P, R, \gamma \rangle$ and policy π
 - ▶ or: MRP $\langle S, P^\pi, R^\pi, \gamma \rangle$
 - ▶ Output: Value function v_π
- Control Problem
 - ▶ Input: MDP $\langle S, A, P, R, \gamma \rangle$
 - ▶ Output: Optimal value function v_*
 - ▶ and: Optimal policy π_*

Iterative Policy Evaluation (1)

- Problem: evaluate a given policy π
- Solution: Iterative application of Bellman Expectation Backup for particular policy π
- Compute: $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_\pi$
- Synchronous backups: All states are updated in parallel
- Initialize $v_0(s) = 0$ for all s
- At each iteration $k + 1$ until convergence
 - ▶ For all states $s \in S$, update $v_{k+1}(s)$ from $v_k(s')$
 - ▶ s' successor state of s
- Convergence to v_π proved

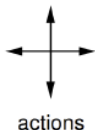
Iterative Policy Evaluation (2)



$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}_k$$

Example: 4 × 4 Gridworld



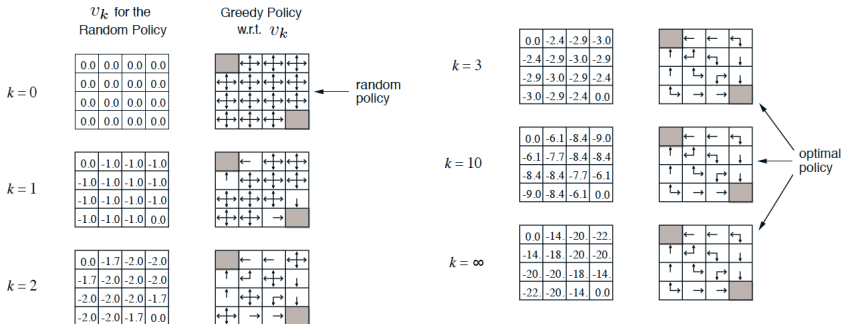
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$r = -1$
on all transitions

- Nonterminal states $S = \{1, 2, \dots, 14\}$
- One terminal state shaded (shown in two places in figure)
- Actions $A = \{up, down, right, left\} \rightarrow$ deterministic state transitions
- Actions that take agent off grid leave state unchanged
- Undiscounted episodic MDP ($\gamma = 1$)
- Reward is -1 until terminal state reached
- Agent follows uniform random policy

$$\pi(u|.) = \pi(d|.) = \pi(r|.) = \pi(l|.) = \frac{1}{4}$$

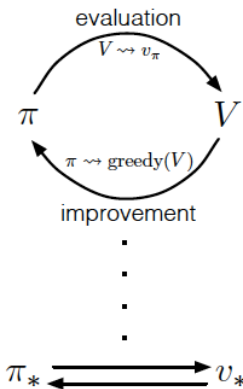
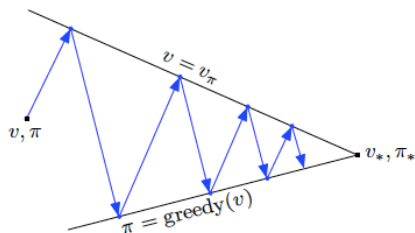
Example: Iterative Policy Evaluation in 4×4 Gridworld



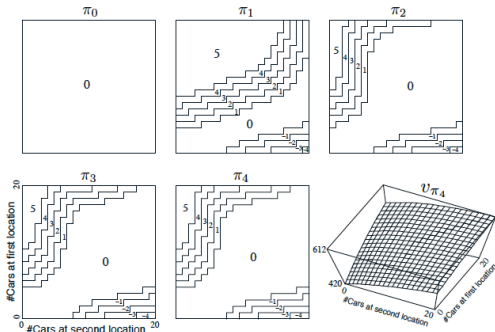
Policy Improvement: How to obtain better policy ?

Policy Iteration

- Policy Evaluation: Estimate v_π
 - ➡ Iterative policy evaluation
- Policy Improvement: Generate $\pi' \geq \pi$
 - ➡ Greedy policy improvement



- States: 2 locations with maximum 20 cars at each location
- Actions: Move maximum of 5 cars between locations overnight
- Reward: 10\$ for each car rented if available
- Transitions: Cars returned and requested randomly
 - ▶ Poisson random process, n returns/requests with prob $\frac{\lambda^n e^{-\lambda}}{n!}$
 - ▶ 1st location: average requests = 3, average returns = 3
 - ▶ 2nd location: average requests = 4, average returns = 2



- $$\pi'(s) = \arg \max_{a \in A} q_{\pi}(s, a)$$

$$q_{\pi}(s, \pi'(s)) \geq \max_{a \in A} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$
$$\begin{aligned}
v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'} [r_{t+1} + \gamma v_{\pi}(s_{t+1}) | s_t = s] \\
&\leq \mathbb{E}_{\pi'} [r_{t+1} + \gamma q_{\pi}(s_{t+1}, \pi'(s_{t+1})) | s_t = s] \\
&= \mathbb{E}_{\pi'} [r_{t+1} + \gamma \mathbb{E}_{\pi'} [r_{t+2} + \gamma v_{\pi}(s_{t+2}) | s_{t+1} = s'] | s_t = s] \\
&\leq \mathbb{E}_{\pi'} [r_{t+1} + \gamma r_{t+2} + \gamma^2 q_{\pi}(s_{t+2}, \pi'(s_{t+2})) | s_t = s] \dots \\
&= \mathbb{E}_{\pi'} [r_{t+1} + \gamma r_{t+2} + \dots | s_t = s] \\
&= v_{\pi'}(s)
\end{aligned}$$

Policy Improvement (2)

- If improvement stops

$$q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

➡ Bellman optimality equation satisfied

$$v_{\pi}(s) = \max_{a \in A} q_{\pi}(s, a)$$

➡ $v_{\pi} = v_{*}$ for all $s \in S$

- π optimal policy

Modified Policy Iteration

- Does policy evaluation need to converge to v_π ?
- Need to introduce a stopping condition ?
 - ▢ ϵ —convergence of value function
- Stop after k iteration of iterative policy evaluation ? ▢ Optimal policy
 - Small gridworld: 3 iterations are sufficient
 - Jack's rental car: 4 iterations are sufficient
- Update policy every iteration i.e. stop after $k = 1$? ▢ Value Iteration

Principle of Optimality

- Any optimal policy can be subdivided into 2 components
 - Optimal first action a_*
 - Followed by optimal policy from successor state s'

Theorem: Principle of Optimality

A policy $\pi(a|s)$ achieves the optimal value from state s , $v_\pi(s) = v_*(s)$, if and only if

- For any state s' reachable from s
- π achieves the optimal value from state s' , $v_\pi(s') = v_*(s')$

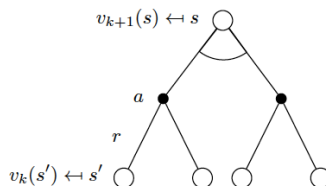
Deterministic Value Iteration

- If we know solution to subproblems $v_*(s')$
- ➔ Solution $v_*(s)$ can be found by one-step lookahead

$$v_*(s) \leftarrow \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_*(s') \right)$$

- Value Iteration: apply these updates iteratively
- Intuition: Start with final rewards and work backwards
- Apply in stochastic MDPs

- Problem: Find optimal policy π
- Solution: Iterative application of Bellman Optimality Backup
- Compute: $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$
- Synchronous backups:
All states are updated in parallel
- Initialize $v_0(s) = 0$ for all s
- At each iteration $k + 1$ until convergence
 - ▶ For all states $s \in S$
update $v_{k+1}(s)$ from $v_k(s')$
 - ▶ s' successor state of s
- Convergence to v_* proved
- No explicit policy unlike policy iteration
- Intermediate value functions may not correspond to any policy



$$v_{k+1}(s) = \max_{a \in A} \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$

$$\mathbf{v}_{k+1} = \max_{a \in A} \mathbf{R}^a + \gamma \mathbf{P}^a \mathbf{v}_k$$

Example: Shortest Path

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

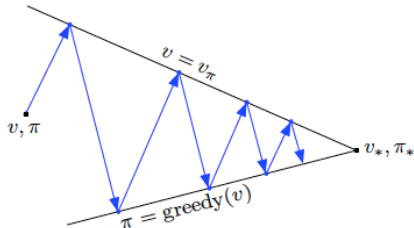
0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

-
- evaluation
- $V \rightsquigarrow v_\pi$
- π V
- $\pi \rightsquigarrow \text{greedy}(V)$
- improvement
- ⋮
- π_* v_*



Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- DP algorithms are based on state value function $v_\pi(s)$ or $v_*(s)$
- Complexity $O(MN^2)$ per iteration for M actions and N states
- Could also apply to state-action value function $q_\pi(s, a)$ or $q_*(s, a)$
- Complexity $O(M^2N^2)$ per iteration

Asynchronous Dynamic Programming

- Synchronous DP: all states are backed up in parallel
- Asynchronous DP: states are backed up individually in any order
 - ▶ Apply appropriate backup for each selected state
 - ▶ Reduce significantly computation
 - ▶ Guaranteed convergence if all states are selected
- 3 simple Asynchronous DP algorithms
 - ▶ In-place DP: Store only one copy of value function for all states
 - ▶ Prioritized sweeping: Use largest Bellman error (value changes) to guide state selection (priority queue)
 - ▶ Real-time DP: Update only states that are relevant to agent

Full-Width vs Sample Backups

- DP uses full-width backups
 - ▶ Consider every successor state and action for each backup
 - ▶ Use knowledge on MDP transitions and rewards
- DP effective for medium-sized problems ($\sim$ million states)
- Bellman's *curse of dimensionality* for large problems
 - ▶ Number of states grows exponentially with number of variables
- Model-free methods use sample backups
 - ▶ Use sample rewards and sample transitions from experience
 - ▶ No need to know MDP model
- Sampling breaks curse of dimensionality
- Constant backup cost independent of states number

